

המכללה האקדמית
להנדסה בתל אביב

אפקה



דו"ח פרויקט גמר

Re - Hospitalization Analysis And Prediction Tool

המחלקה למדעי המחשב מכללת אפקה

מנחה: ד"ר שרון ילוב – הנדזל

מגישות:

מאיה זגדון, תז: 206572976

אופיר בן מנשה, תז: 316123991

תודות

ברצוננו להודות לד"ר שרון ילוב הנדזל שהנחתה אותנו לאורך העבודה המתמשכת על הפרויקט, שקישרה אותנו לעבודה עם בית החולים, שתמכה בנו, הסבירה ועזרה לנו בסבלנות ואדיבות.

התמיכה, השאיפה למצוינות, המשוב והעזרה הרבה לאורך הפרויקט סייעו רבות בעיצוב הפרויקט שלנו.

ברצוננו להודות גם למרכז החדשנות של בית חולים מאיר על המידע והתמיכה שניתנה לנו במסגרת הפרויקט ובפרט ברצוננו להודות לד"ר גיא טופז על ההיענות לשאלות, שיתוף הפעולה והמסירות לפרויקט.

Table of contents

executive summary	4
project's goal.....	5
motivation.....	5
objectives.....	5
metrics.....	6
literature review.....	7
competitors	12
functional requirements.....	14
non-functional requirements.....	15
system's functionality.....	16
use cases diagrams.....	21
detailed design.....	31
architecture.....	31
data description.....	33
API specifications.....	34
interface design.....	35
programming language and tools.....	35
user interface description.....	39
algorithm description.....	44
system's versatility.....	47
instruction.....	50
summary and conclusions.....	52
known issues	54
bibliography.....	55

Executive summary

Our project is a system that helps in carrying out research and predictions related to the subject of re-hospitalizations at Meir Hospital.

The issue of re-hospitalizations is a well-known phenomenon in Israel and the world, the phenomenon incurs many costs to the health systems of the world every year.

The purpose of our system is to provide an efficient and objective tool for researching the issue and to allow the hospital to make more informed decisions based on the information revealed from the research.

Our work included an extensive literature review on the topic of re-hospitalizations, we studied the phenomenon in depth, its causes, and its effects.

We found several studies analyzing the factors influencing the trend of increasing re-hospitalizations and ways of dealing with them using artificial intelligence.

After that, we reviewed competitors, and although we found several advanced systems for analyzing and researching information using artificial intelligence. We came to the conclusion that our system would be the most suitable for the research at Meir Hospital because it was custom-made according to the type of information provided by the hospital and was created to meet their needs. We have made it our goal to be a worthy competitor to the other existing systems in the market and to provide the hospital with a reliable, cheap system that meets exactly their needs for research purposes.

Finally, we developed a system consisting of two main parts, the first deals with the visualization and accessibility of the information about the re-hospitalizations in a visual way and the second part deals with the creation of prediction by three types of artificial intelligence models.

Project Goal:

Our main goal is to develop a tool for analyzing and comparing information on hospitalized patients at Meir Hospital and predicting readmissions using Machine learning models in different techniques (ML).

Motivation:

Readmissions are a global problem that hospitals face.

In Israel, 1 in 8 hospitalized patients return for readmission within a month. Readmission to the hospital is a process in which a patient is discharged from the hospital and readmitted within a particular time, which we will define for the project as six months since the previous discharge. Readmission is one of the most expensive treatments in the United States, costing \$41.3 billion annually.

Objectives:

- Development of a program that collects patient data and displays information trends in various graphs according to criteria selected by the user.
- To present the different algorithms for training the model and allow the user to choose the desired algorithm. The available options are Decision trees, K-nearest neighbors, and SVM. We will apply different ML methods to perform classification on readmissions.
- To run a trained model on new data. The user can choose a trained model, load new data, and run the trained model on this data.

To answer the most important question: what are the factors that influencing the re-hospitalization rates in Meir hospital? .

Metrics:

- When the graph generator is able to generate a graph according to the parameter chosen.
- Successful training of a model and saving it according to the user's request.
- When the model can find logical and rational predictions according to the type of algorithm chosen.
- Presenting the ML predictions with an explanation and a visual option to see the data on which it is based.
- When the algorithm is able to predict metrics with at least 50% of success.
- When the filter process shows the right information.
- When the user is able to update the CSV file – add or remove a certain field and the system should support it. (nice to have).

Literature Review:

Our project deals with data analysis using statistical tools, drawing conclusions, and making hypotheses from the information using a machine learning-based tool.

Our ultimate goal is to provide an ML-based tool that can predict readmissions and focus on risk groups for hospitals.

Using the tool we build, we want to inform the hospital about the probability that a particular patient will return for readmission to an internal medicine department and make hypotheses about the relationship between the different data.

In Israel:

According to an article published in November 2022, 1 in 8 hospitalized patients return for readmission within a month. The article reviews a report of the general hospitalization departments over a decade from 2010 to 2020.

One out of eight hospitalized patients (12.5%) returned for readmission, meaning they were discharged from a previous hospitalization within the last month—a decrease from 13.4% in 2019.

Readmissions measure the quality of care in the first hospitalization and may indicate a "miss" in diagnosis, problems with discharge to community clinic follow-up, or trouble in medication administration.

According to the report, in 2020, 1.3 million hospitalizations in Israel lasted for 4.5 million days. More than a quarter of the days of hospitalization (27%) were in the internal medicine departments.

According to an article by the Ministry of Health on the subject of hospitalizations in internal medicine departments in particular, the following data emerges:

Readmissions: In 2019, 19% of hospitalizations in internal medicine departments were readmissions within a month and 8% within a week, a relatively stable rate since 2005. The rate increases with age; 23% of hospitalizations of people aged 85+ were readmitted within a month, 21% aged 75-84, 19% aged 55-74, 15% aged 45-54, and 11% aged 15-44. The percentage is higher the longer the length of stay in the baseline hospitalization, 27%-24% for patients who were eight days or more in the baseline hospitalization, compared to 18%-15% for patients who were 2-4 days in the baseline hospitalization.

Factors Affecting Readmission:

This section discusses the factors that were found to significantly impact readmission within 30 days, according to a multivariate model that examined the characteristics of hospitalized patients and hospitalization in 2019.

- **Age:** The probability of readmission increases with age.

Patients aged 85 and over have a higher chance of readmission (1.6 times), followed by patients aged 75-84 (1.4 times), 55-74 (1.3 times), and 45-54 (1.2 times) compared to patients aged 15-44.
- **Gender:** Men were found to be 1.06 times more likely to be readmitted than women. This finding suggests that men may be at a higher risk for certain health conditions or complications.
- **Hospitalization in the previous six months:** Increases the chances of readmission by 2.4 times compared to other cases.
- **Urgent admission:** Increases the chances of readmission by 1.9 times compared to elective patients.
- **Length of stay:** The chances of readmission increase with the longer stay in the baseline hospitalization. A stay of 15 days or more increases the chances of readmission by 1.3 within 30 days, a stay of 8-14 days increases the chances by 1.2, and a stay of 2-4 days decreases the chances to 0.8 compared to a stay of 5-7 days in the baseline hospitalization.

- **Day of the week:** Patients discharged on Sunday have a lower chance of readmission (0.94).
- **Month of discharge:** No significant difference was found in monthly readmission rates, except for a 1.07 increase in June compared to May.
- **Hospital size:** Hospitals with 200 or more internal beds have a 1.06 times higher chance of readmission, and hospitals with less than 100 beds have a 1.15 times higher chance of readmission than hospitals with 100-199 beds.
- **Health insurance fund:** Members of Maccabi and Meuhedet have a lower chance (0.9) of readmission than members of Clalit. No significant difference was found for members of Leumit compared to Clalit members.

Comorbidities: Patients with metastatic malignant tumors have a 1.9 times higher chance of readmission, and patients with non-metastatic cancer have a 1.3 times higher chance compared to patients without these diseases.

Patients with liver failure have a 2.2 times higher chance of readmission compared to patients without this disease, and patients with liver disease have a 1.7 times higher odds ratio for readmission. Patients with heart failure or dementia have a 1.4 times higher chance of readmission. Patients with kidney failure or COPD have a 1.3 times higher chance of readmission compared to patients without these diseases. Patients with dementia and diabetic patients with complications have a 1.2 times higher chance of readmission compared to patients without these diseases. Diabetic patients without complications, patients with heart attacks, and patients with peripheral vascular diseases have a 1.1 times higher chance of readmission compared to patients without these diseases.

Considering the direct influence of these diverse factors on readmission rates, they will all be incorporated into the analysis of Meir Hospital's internal medicine departments and the subsequent conclusions drawn.

Specifically, readmissions at Meir Hospital in 2019:

- Fourteen thousand people were hospitalized
- Of those, 14% were hospitalized twice.
- 8% were hospitalized three times.

Readmission Rates in the United States:

The annual cost of readmissions within 30 days is \$41.3 billion, with readmission rates ranging from 11.3% to 22.3%.

Readmission is one of the most expensive medical procedures in the US.

The high annual costs indicate inadequate hospital quality and hinder resources available for other government programs that require attention and erode the industrial competitiveness of the US.

Preventing Readmissions with Artificial Intelligence:

The article "Implementation of Artificial Intelligence" discusses a research study on AI tools and how they contribute to hospitals. The tool is designed to reduce readmissions by drawing conclusions about patients, making recommendations to prevent readmission, and focusing on risk groups. The research concludes that the use of artificial intelligence led to a decrease in readmissions.

Hospitals are looking for different strategies to prevent readmissions and try to predict them. One strategy is to identify patients at the highest risk for readmission and focus resources and attention on this risk group.

The article shows that focusing treatment on patients at high risk for readmission can save significant financial costs for hospitals and patients.

Various models are used to predict readmission to the hospital, including:

Logistic regression, Decision trees, and One widely used predictive model is LACE. LACE considers length of stay, comorbidities, and emergency department use to predict 30-day readmission to the hospital.

However, this model has been criticized because it may not apply to specific populations and may provide little added value compared to clinical judgment alone.

Nevertheless, the study results showed a significant decrease in readmission rates from 11.4% to 8.1%.

Many studies have concluded that AI-based models are superior to traditional risk stratification models. **Therefore, our project will use ML algorithms that have proven effective and better suited for this purpose.**

Competitors:

category	our project	Polymer	Tableau	Microsoft BI	MediData	R
price	free	50\$ minimum	75\$	1\$-193\$	unknown.	free
support	Telephone assistance	helpdesk	helpdesk	helpdesk	During installation, there is a representative; after that, it depends on the route.	none
Displaying statistics	yes	yes	yes	yes	yes	yes
Use of AI	yes	yes	yes	yes	yes	no
Adaptation to the world of medicine	yes	No	no	no	yes	no
Generate graphs automatically from the data	yes	yes	yes	yes	yes	no
Characterization of relationships between different factors	yes	yes	yes	yes	yes	no

Our Advantage Over Competitors:

Our main advantage over other competitors is our ability to customize our solution specifically to the needs of Meir Hospital. This includes building functionality and designing the UI according to Meir Hospital's requirements.

We are specifically focused on the healthcare industry, while our competitors in the market are more focused on financial data analysis.

We offer the same services as our competitors but at no cost.

Functional Requirements:

System Requirements:

- The user selects the factors displayed in the graph.
- The user can select one factor for each graph and no more or one factor and one filter.
- The system will be able to transfer the new data uploaded by the user to the database.
- The system presents the user with several options for the type of criteria.
- After the user chooses the type of criteria, the system generates a graph according to the user's choices.
- The system will support a variable length of information fields about patients.
- The system will give the user the option to generate a graph according to the filter selected.

Model Requirements:

- Training a model based on different algorithms and storing it in the database.
- The model will be able to make predictions due to the data uploaded by the user.
- The model will be able to predict whether a patient will return for readmission and how long will it take until they will return.

Database Requirements:

- The database holds several trained models.
- A database that holds information about re hospitalization and ER load data.

Non-Functional Requirements:

Design Requirements:

- A straightforward and user-friendly interface allows new users without a technological background to operate the application themselves from the first use.
- Easy design to prevent user confusion.

Performance Requirements:

- The system can create a requested graph according to the selected criteria in a minimal time of 2 seconds.
- We will define a specific level of accuracy that we will require from our model for prediction making ~ 30% accuracy rate.

Operational Requirements:

- The system can interface with a CSV file and read data from it according to the format provided by the hospital.
- The system can adjust itself to the changes made in the file – in cases of adding or removing a field. (nice to have).
we will elaborate on the necessary changes needed to be done in order to support other files.

System's functionality

Files uploading-

1. ER data upload
2. Readmission data upload

In the initial use of the system, the user must provide those 2 files in a CSV / xls format. **Without those files, the system won't allow the continuation of use.**

Graph plotting –

Our system is able to generate bar graphs, each graph has multiple categories on the x-axis, and the amount on the y-axis.

The graphs our system can generate:

1. Diagnoses at admission for re-hospitalization –
Once the user picks this option, a list of all the diagnosis groups opens and the user is required to pick one group (respiratory for example),
Then all the diagnoses in this group are presented in a graph.
2. Admitting doctor
3. Diagnoses at ER - Once the user picks this option, a list of all the diagnosis groups opens and the user is required to pick one group (respiratory for example),
Then all the diagnoses in this group are presented in a graph.
4. Cause of contacting triage
5. Duration of staying in ER (in hours)
6. Date and time of admission – once the user picks this option, a sub-menu is opened, and the user is required to pick one option:

- Sort by hours
- Sort by the day of the week
- Sort by months
- Sort by years

Once the user picks an option, the desired graph is generated.

7. Department of first admission

8. Time (in days) since the first admission to the second one

9. Diagnoses at discharge from the first hospitalization- Once the user picks this option, a list of all the diagnosis groups opens and the user is required to pick one group (respiratory for example),
Then all the diagnoses in this group are presented in a graph

10. Time and date of release – once the user picks this option, a sub-menu is opened, and the user is required to pick one option:

- Sort by hours
- Sort by the day of the week
- Sort by months
- Sort by years

Once the user picks an option, the desired graph is generated.

11. Admission duration

12. Department of readmission (department 6 in file)

13. Age

14. ER load – once the user picks this option, a sub-menu is opened, and the user is required to pick one option:

- Sort by hours
- Sort by the day of the week

- Sort by months
- Sort by years

Once the user picks an option, the desired graph is generated.

Filtered graph plotting –

Our system is able to generate bar graphs generated from 2 categories from files. One serves as the filter and the other as the data source.

For example-

filter: department A, data: ages. So, the graph will be generated is made from all ages of patients admitted in department A.

each graph has multiple categories on the x-axis, and the amount on the y-axis.

Filters:

1. Diagnoses at admission for re-hospitalization – Once the user picks this option, a list of all the diagnosis groups opens and the user is required to pick one group (respiratory for example),
Then all the diagnoses in this group are presented in a graph.
2. Admitting doctor – the user will required to choose a doctor from the list.
3. Diagnoses at ER - Once the user picks this option, a list of all the diagnosis groups opens and the user is required to pick one group (respiratory for example),
Then all the diagnoses in this group are presented in a graph.
4. Cause of contacting triage – the user is required to choose one diagnose from the list.

5. Department of first admission - the user required to choose a department.
6. Diagnoses at discharge from the first hospitalization- Once the user picks this option, a list of all the diagnosis groups opens and the user is required to pick one group (respiratory for example),
Then all the diagnoses in this group are presented in a graph
7. Department of readmission (department 6 in file) - the user required to choose a department.

When the user applies a filter, a comprehensive list of parameters becomes available for selection. The goal is to offer a wide range of options, although certain graphs may not be feasible to create due to logical constraints. Therefore, the choice of parameters is dependent on the filter applied.

Train machine learning model –

Our system supports 3 types of machine learning algorithms:

1. Decision tree
2. Support vector machines
3. K nearest neighbours

The user prompt to pick one algorithm for training the model.

Model training

The training process is done on the following features:

- Age
- Admission duration time (in days)
- Time of discharge

And the target is: time (in days) from discharge to readmission.

The target is what we trying to predict with the models.

The user has the option to filter the age feature and train the model only on features that matches the age range selected.

If no filter is selected, the model will be trained on the full data.

After a model is trained according to the user's selection, the user may watch the model's scores by clicking the "show evaluation results" button.

Evaluation displays the following data:

- Accuracy
- Precision
- Recall
- F1 score

Make machine learning prediction

After a model has been trained, it can be utilized to predict the duration between the first and second hospital admissions for a new patient. To make this prediction, the system requires the input of specific patient data, including:

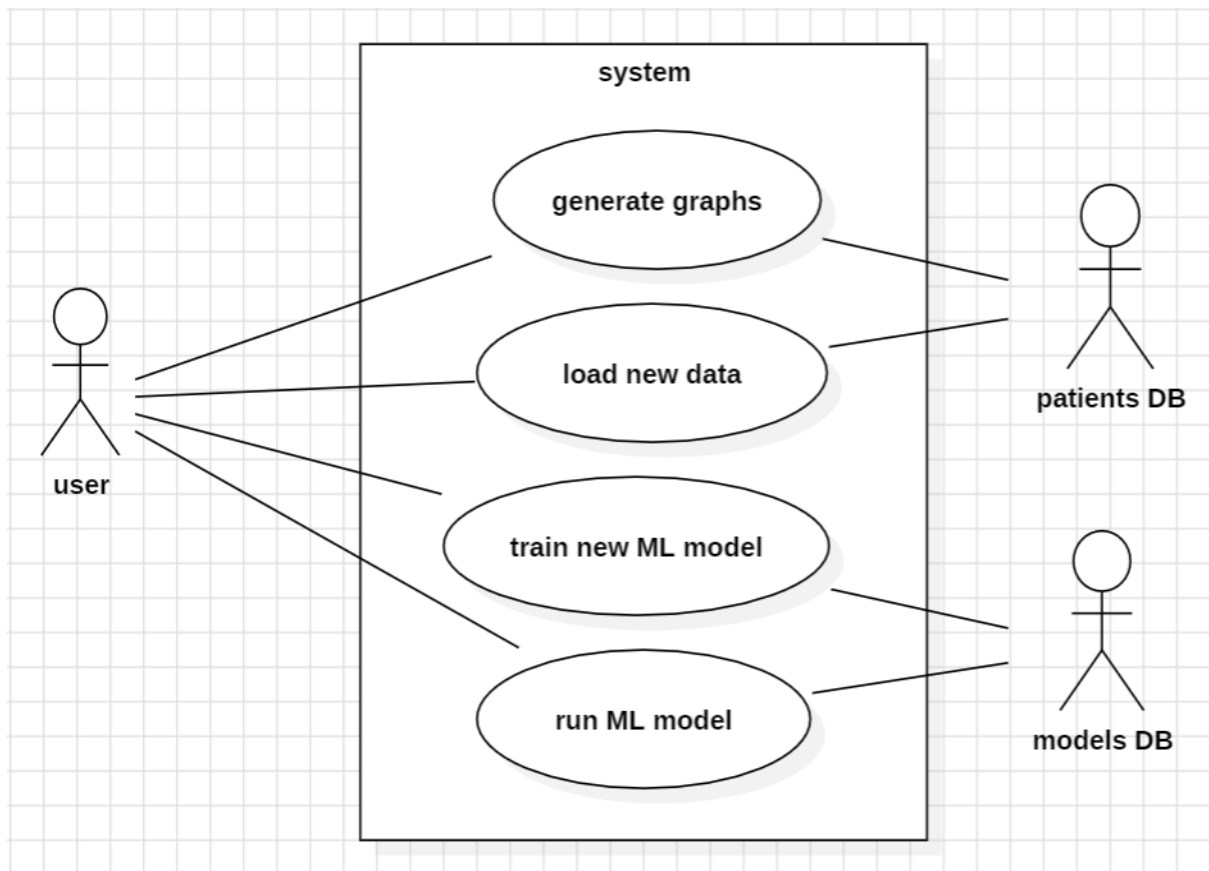
- Age: The patient's age at the time of the first admission.
- Time of Discharge: The time or date when the patient was discharged from their first admission.
- Admission Duration: The total number of days the patient was admitted during their first hospital stay.

Once these details are entered into the system, the trained model processes the information and predicts the likely time interval before the patient might be readmitted. The output is a categorization or estimation of the time frame that best matches the patient's situation.

The groups:

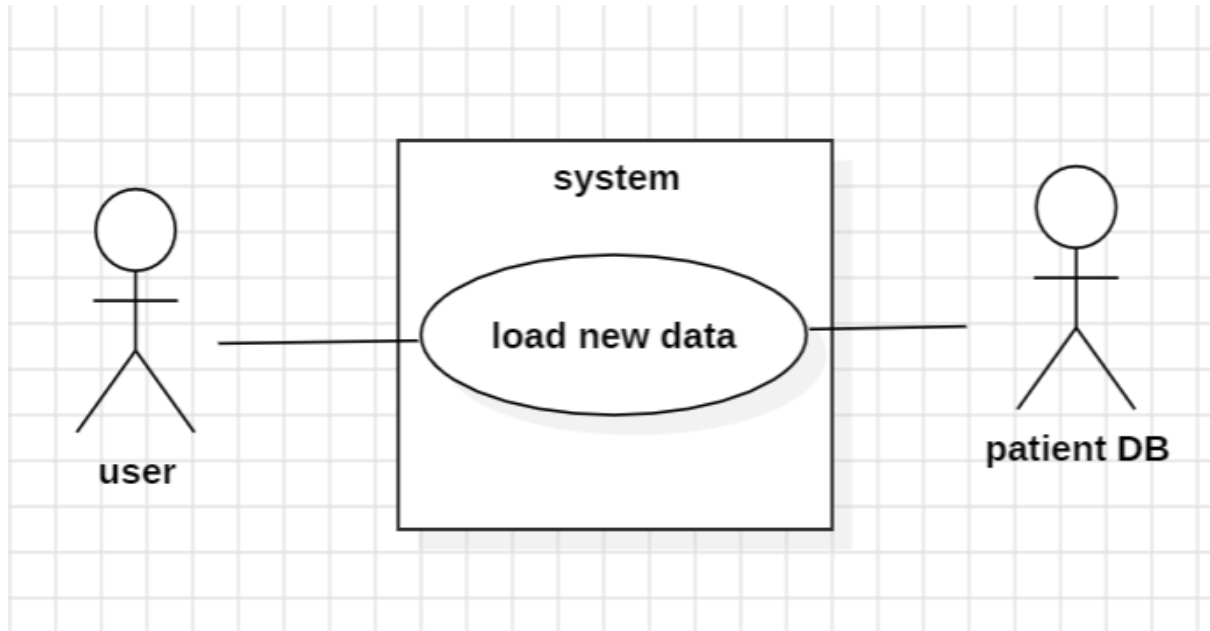
- Group 0: 1-10 days
- Group 1: 11-20 days
- Group 2: more than 20 days.

Use Case Diagram - Initial Functional Analysis



Use cases

Loading New Patient Information



Main Flow:

1. The user selects the "Upload File" option from the menu.
2. The system prompts users to upload a CSV file containing hospitalized patient data.
3. The system saves the data to the hospitalized patient database.
4. The system checks if the new data entered already exists in the system.
5. The system records the new data in the patient database.

Secondary Flow A:

This secondary flow diagram follows step 2 in the main flow.

2A1. The file is not uploaded or not opened correctly by the system.

2A2. The system sends the user a message about the error and asks the user to try again or upload a different file.

2A3. If the new upload is successful, we proceed to step 3 in the main flow. Otherwise, we return to step 1.

Secondary Flow B:

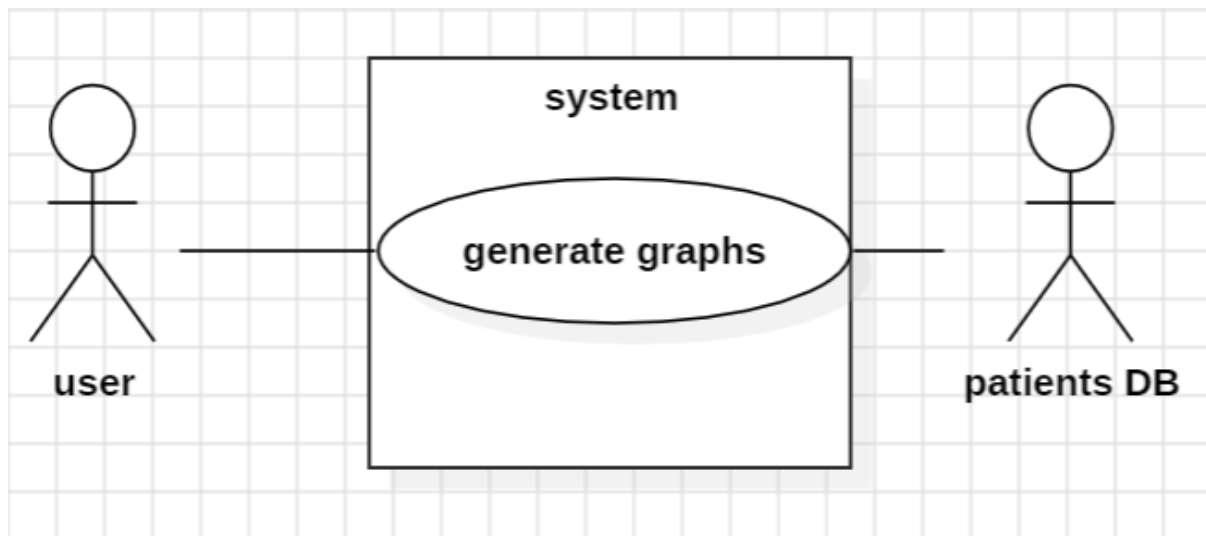
This secondary flow diagram follows step 2 in the main flow.

2B1. The file is not a CSV file or not supported by the system formatting requirements.

2B2. The system sends the user a message about the error and asks the user to try again or upload a different file.

2B3. If the new upload is successful, we proceed to step 3 in the main flow. Otherwise, we return to step 1.

Generate Graph



Main Flow:

1. The user selects the "Generate Graph" option from the menu.
2. A menu opens, displaying the data fields available in the patient database.
3. The user selects one field.
4. The system extracts the desired information from the patient database.
5. The system generates a graph based on the user's selections.

Secondary Flow A:

This secondary flow diagram follows step 3 in the main flow.

- 3A1. The user doesn't select a field.
 - 3A2. An error message is sent to the user, and another attempt is made.
- We return to step 4 in the main flow.

Secondary Flow B:

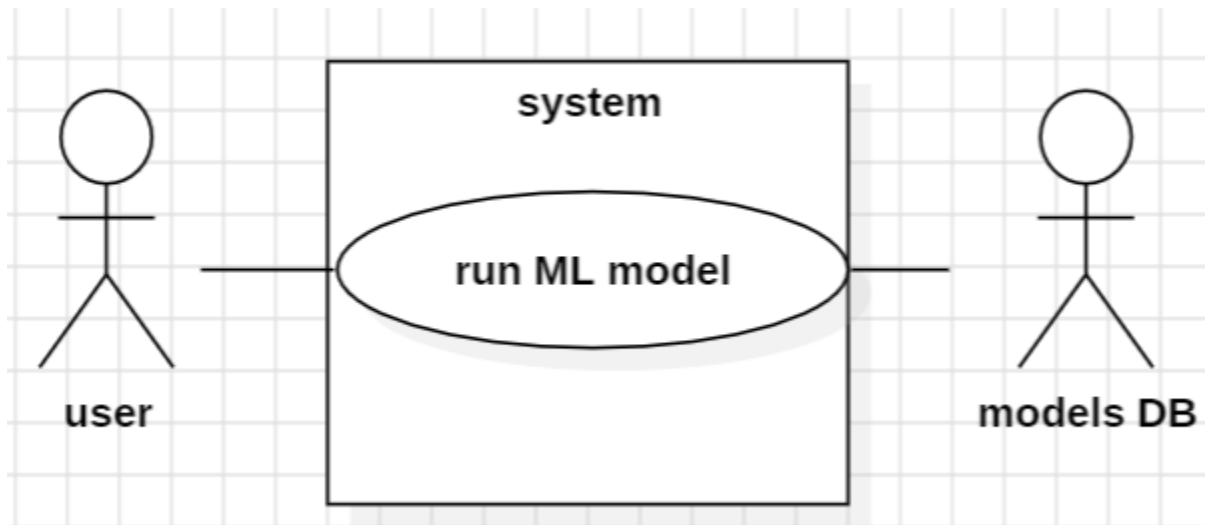
This secondary flow diagram follows step 3 in the main flow.

3B1. There is no patient information in the database.

3B2. An error message is sent to the user, and a message is sent to the user to upload a data file.

3B3. We proceed to the new data upload scenario.

Run Machine Learning Algorithm



Main Flow:

1. The user selects run machine learning algorithm.
2. The user required to enter a data about a new patient in order to make the prediction.
3. A list of trained models stored in the database opens.
4. The user selects a trained model.
5. The algorithm runs on the new data according to the model type selected.

Secondary Flow A:

This secondary flow diagram follows step 3 in the main flow.

- 3A1. There are no trained models stored in the database.
- 3A2. A message will prompt to proceed to the model training scenario.

Secondary flow B:

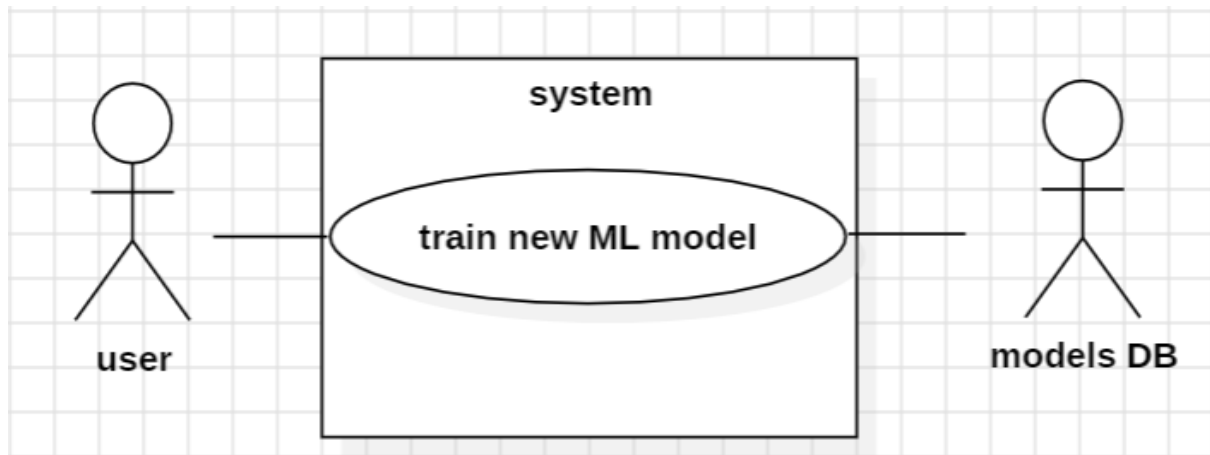
This secondary flow diagram follows step 2 in the main flow.

2B1. User load invalid patient data.

2B2. Systems displays an error message and request the user to try and load a different file.

2B3. Getting back to step 2 of the main flow.

Train New Model



Main Flow:

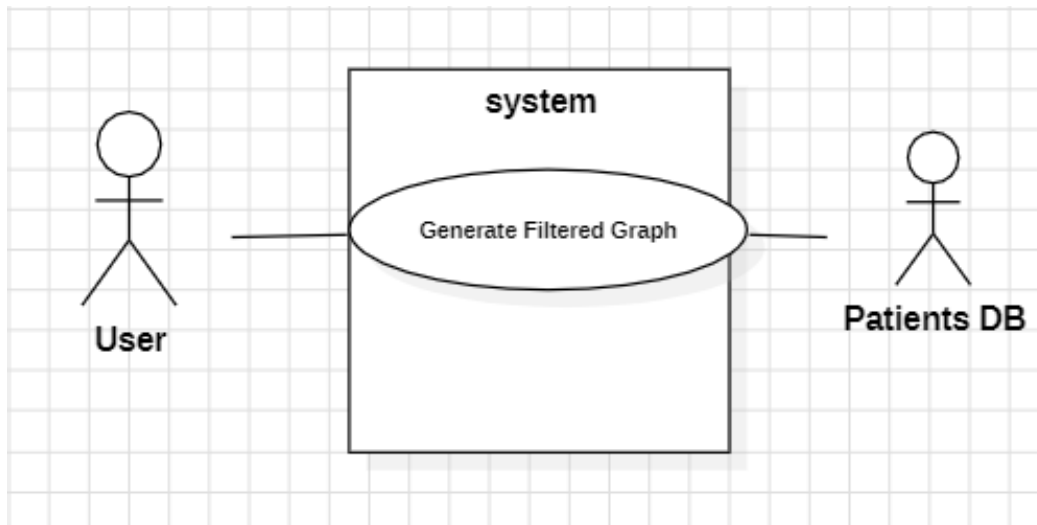
1. The user selects the "Train Model" option from the menu.
2. A list of possible algorithms for training the model is displayed.
3. The user selects an algorithm to train the model.
4. The system trains the model according to the selected algorithm.
5. The system saves the trained model in the model database.

Secondary Flow:

This secondary flow diagram follows step 4 in the main flow.

- 4A1. Model training fails.
- 4A2. The system sends the user the reason for the failure and asks for another attempt - we return to step 3.

Generate Filtered Graph



Main Flow:

1. The user selects the "Create Filtered Graph" option from the menu.
2. A menu opens, displaying the data fields available in the patient database.
3. The user selects one field.
4. The user sets a filter according to the field he chose.
5. The system extracts the filtered information from the hospitalization database.
6. A menu opens, displaying the data fields available in the patient database.
7. The user selects one field as a parameter shown in graph.
8. The system generates a graph based on the user's selections.

Secondary Flow A:

This secondary flow diagram follows step 3 and 6 in the main flow.

- 3A1. The user selects too few or too many fields.
 - 3A2. An error message is sent to the user, and another attempt is made.
- We return to step 4 or 7 accordingly in the main flow.

Secondary Flow B:

This secondary flow diagram follows step 3 in the main flow.

3A1. There is no patient information in the database.

3A2. An error message is sent to the user, and a message is sent to the user to upload a data file.

3A3. We proceed to the new data upload scenario.

Detailed Design

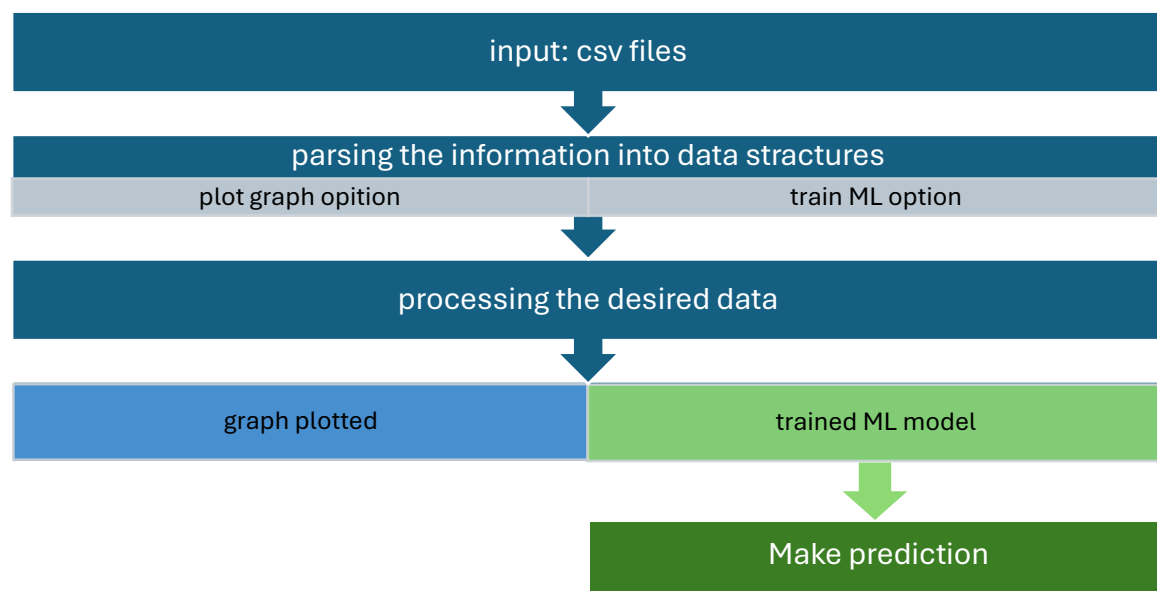
Architecture-

The architecture best suits our project is **MVC**.

General explanation of the architecture:

The system reads the desired information from the selected files. It parses the information into data structures, essentially undergoing a process of data processing. Afterward, the user is given two options: to create various graphs or to train a model on the data.

Chart Flow explanation:



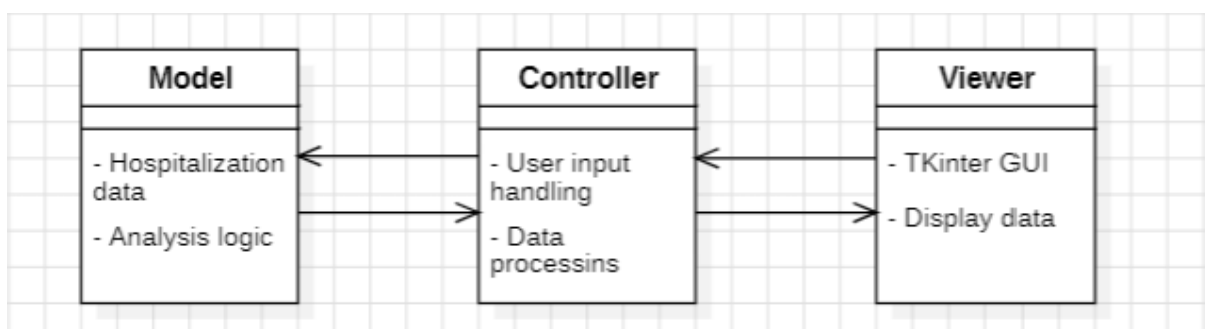
MVC Architecture in our project:

- **Model:**
 - Manages hospitalizations and patient data, trained ML data, and other relevant information.
 - Handles data retrieval, storage, and processing.
- **View:**
 - GUI created using Tkinter.
 - Presents data to hospital staff.
 - Accepts user input.
- **Controller:**
 - Acts as a bridge between the Model and the View, mostly automated by Tkinter.
 - Handles user input, processes it, and updates the Model and View accordingly.

Data Storage:

- The hospitalization list and data on patients are stored in CSV.
- Trained ML models will be stored in files.
- Other relevant data is stored in separate files.

Graphic Description



data description:

- **Hospitalizations list** - a CSV file that has the following fields:
 - serial number
 - age
 - department
 - hospitalization duration
 - date and time of release of the patient
 - diagnose of discharge from the first hospitalization
 - Time (in days) from the first discharge to the next hospitalization
 - department
 - date and time of the second hospitalization
 - duration time in ER
 - Reason for contacting triage
 - Diagnoses in the emergency room
 - Hospitalist (doctor)
 - Diagnoses at admission for re-hospitalization

- **ER rates list** - a CSV file that has the following fields:
 - date of arrival
 - day of the week
 - walk-in clinic ER
 - internal ER
 - Infectious emergency medicine

- **Diagnose list** - a CSV file that contains all possible diagnoses for hospitalization. The diagnoses divided to the following groups:
 - Respiratory
 - General
 - Cardiovascular
 - Infectious Diseases
 - Fluid and Electrolyte Imbalances
 - Neurological

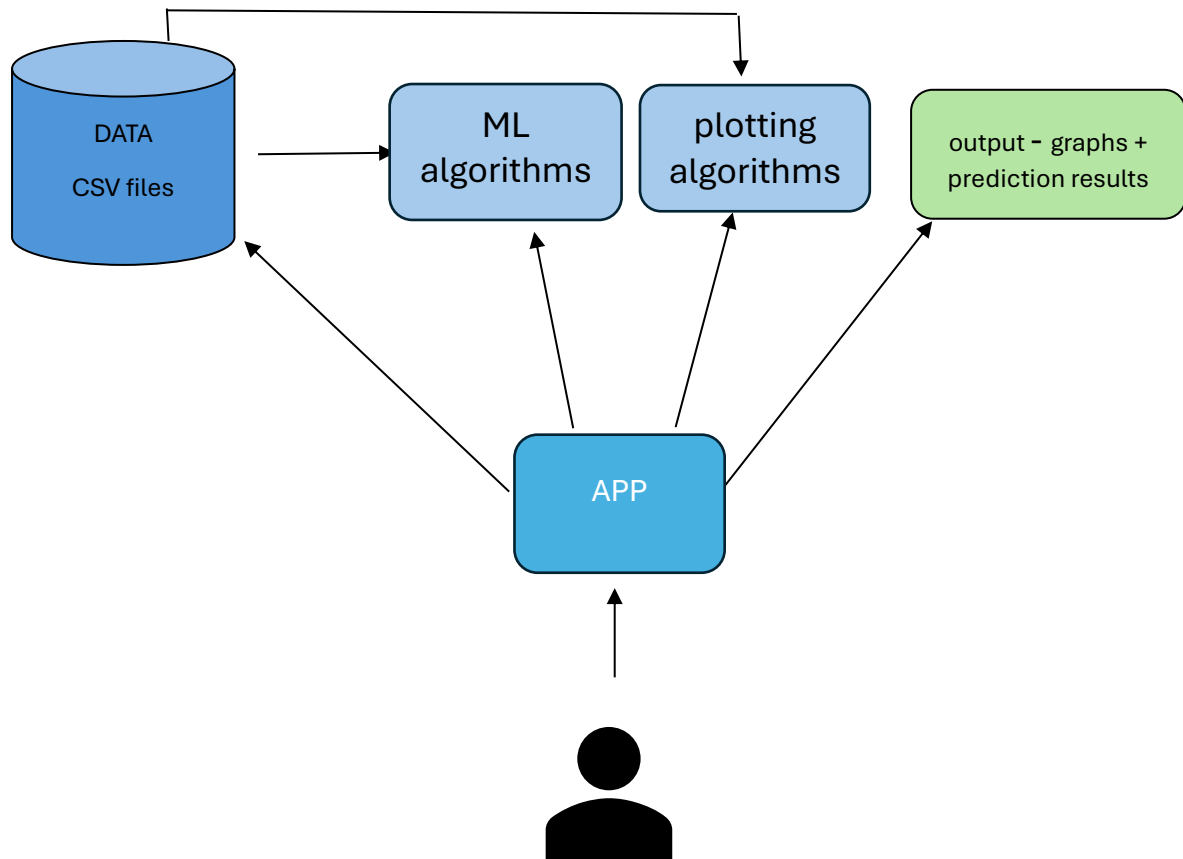
- Renal
- Metabolic Disorders
- Gastrointestinal and digest Disorders
- Hematological
- Dermatological Conditions
- Urinary
- Narcotics
- Liver Diseases
- Trauma
- Musculoskeletal Disorders
- Vascular Disorders
- Oncology
- Mental and Behavioural Disorders

API specification:

User interface:

- Upload CSV of readmission hospitalization data.
- Upload CSV of ER rates data.
- Upload CSV of diagnoses data.
- choose a parameter for the graph generator - for one parameter graphs.
- choose parameters for the filtered graph generator.
- Show a graph.
- Save graphs as png.
- Choose an algorithm for training ML.
- Prediction results of the ML model display.
- Save an ML model.

interface design:



Programming Languages and Tools –

We chose Python for our project due to its robust data analysis capabilities, which are supported by a wide array of libraries. Additionally, we required a responsive user interface, which we achieved by incorporating the TkInter library into our project.

We will elaborate on the use of each library on the following page.

Libraries used in the project and their applications:

1. **Tkinter:** Tkinter is the standard GUI (Graphical User Interface) library for Python. It provides a fast and easy way to create simple GUI applications. Tkinter is built on the Tk GUI toolkit and provides a way to create windows, dialogs, buttons, labels, and other GUI widgets.

In our project: Serves as the GUI framework for the application

2. **Pandas:** Pandas is a powerful data manipulation and analysis library for Python. It provides data structures like Series and DataFrame, which are designed to work with structured data very easily and intuitively. Pandas is commonly used for data cleaning, exploration, and analysis.

In our project: Utilized for data manipulation and analysis. It is used to read the CSV files, clean data, and prepare it for machine learning.

3. **JSON:** The json module is a part of the Python standard library used to work with JSON data. JSON (JavaScript Object Notation) is a lightweight data interchange format that is easy for humans to read and write, and easy for machines to parse and generate. The json module provides methods to parse JSON strings and files into Python objects, and to convert Python objects back into JSON format.

In our project: we use the json for configuration data such as file paths, diagnosis data and other settings.

4. **os:** The os module in Python provides a way of using operating system-dependent functionality. It allows you to interface with the underlying operating system that Python is running on, and executing system commands.

In our project: used it to navigate file paths, and manage directories.

5. **Matplotlib:** Matplotlib is a plotting library for Python which provides an object-oriented API for embedding plots into applications. It is widely used for creating static, interactive, and animated visualizations in Python. Matplotlib can be used in Python scripts, the Python and

IPython shell, web application servers, and various graphical user interface toolkits.

In our project: This library used to create visualizations such as bar graphs. It provides a variety of functions to customize the appearance of the plots.

6. **Pickle:** The pickle module is used for serializing and deserializing Python object structures, also called marshalling or flattening. Serialization refers to the process of converting a Python object into a byte stream to save it to a file/database, or send it over a network, and deserialization is the reverse process.

In our project : it used for saving and loading trained machine learning models. This allows the application to persist models to disk and load them later for making predictions without needing to retrain.

7. **sklearn:** Also known as scikit-learn, this library is one of the most popular machine learning libraries for Python. It features various algorithms for classification, regression, clustering, and dimensionality reduction. It also includes tools for model fitting, data preprocessing, model selection, and evaluation.

In our project: used for machine learning. It provides tools for model fitting, preprocessing, cross-validation, and various machine learning algorithms.

8. **re:** The re module provides regular expression matching operations similar to those found in Perl. Regular expressions are a powerful language for matching text patterns. This module allows Python to use regular expressions to search text for patterns, extract information, or replace substrings.

In our project: used for working with regular expressions. also text processing, such as extracting patterns from strings, validating input, or cleaning data.

9. **numpy**: NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays. It is the foundational library for scientific computing in Python and is used for performing numerical computations.

In our project: used for numerical operations on data within the project.

10. **imblearn**: Short for imbalanced-learn, this library is an extension to scikit-learn and is used for dealing with imbalanced datasets. It provides various methods for under-sampling and over-sampling to adjust the class distributions for imbalanced datasets. It's useful in scenarios where the number of observations in one class is significantly lower than those of the other classes.

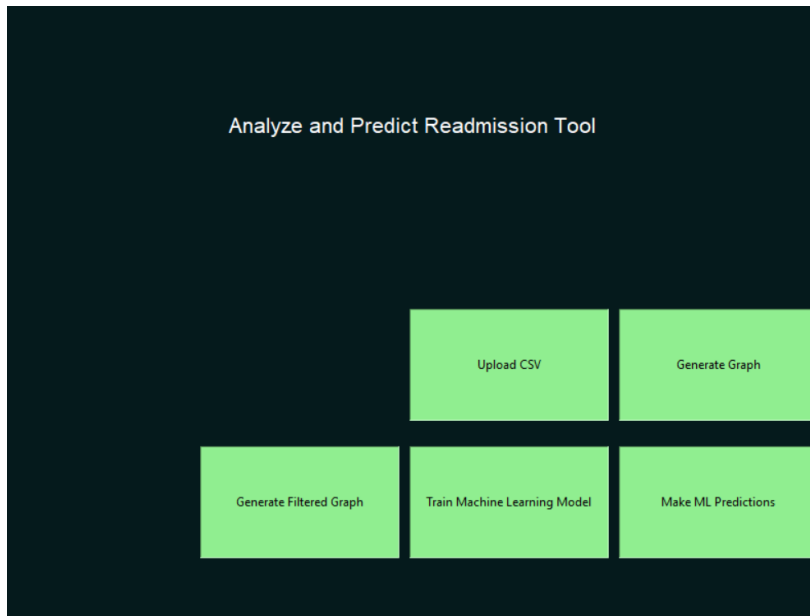
In our project: utilized to address class imbalance during the training of machine learning models. responsible for randomly reducing the number of examples in the majority class to balance the dataset before training the classifier.

11. **Openpyxl** - is a library used for reading and writing Excel 2010 xlsx/xlsm/xltx/xltn files. It is designed to work with the open XML format used by recent versions of Microsoft Excel. The library allows users to automate Excel tasks or handle Excel files in a programmatic way without having to use Excel itself.

In our project: used for reading xlsx/xlsm/xltx/xltn files.

User Interface Description

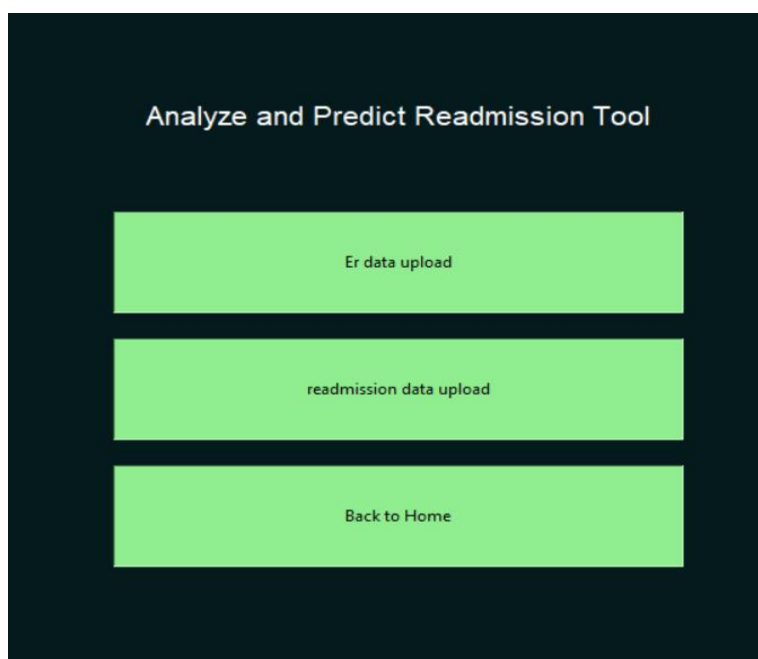
Enter page –



as in the image, the user may choose the desired action by pressing the matching button.

There are 5 actions the user can choose from.

1. Upload CSV –



When the user clicks the "ER data upload" button or the "readmission data upload", a file dialog will appear, allowing the user to navigate through their file system and select a file.

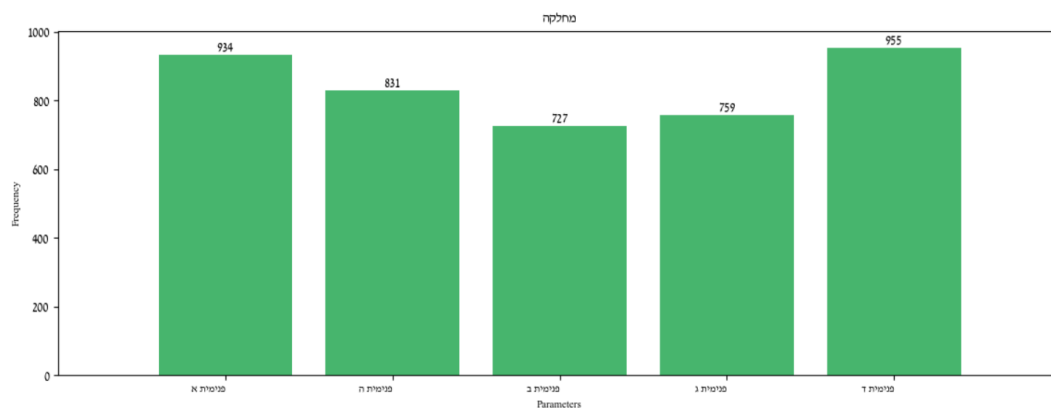
2. Generate Graph-



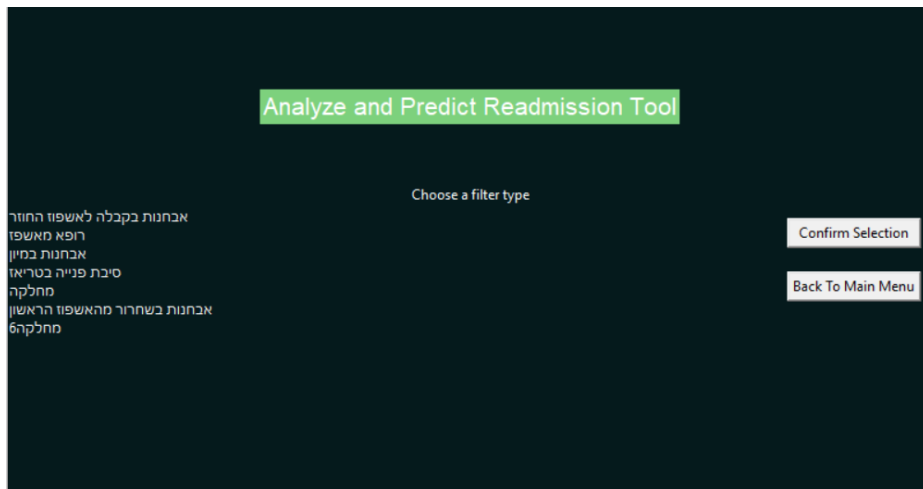
As shown in the image, lines with a matching field name from the CSV will be presented on the screen. The lines are reactive and respond to the user's click.

Once the user presses one of the lines and presses "confirm selection", the system will generate a graph according to the user selection.

An example graph –



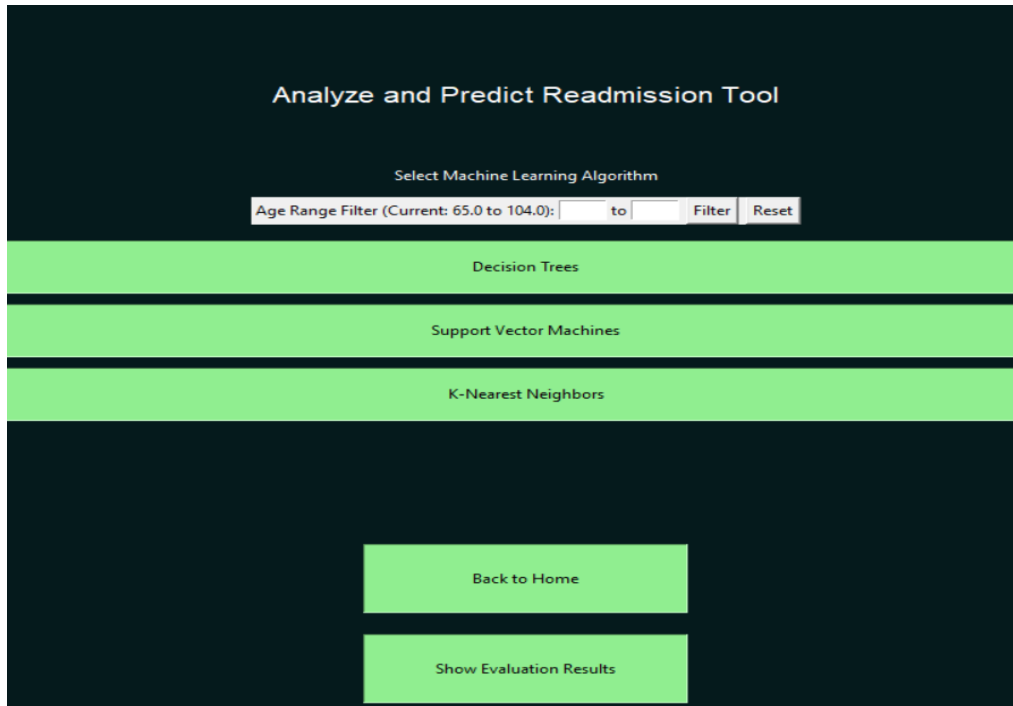
3. Generate Filtered Graph-



After the user chooses one filter category the system will display the different parameters in this filter. the user will need to choose one filter.

Once the user chooses a filter, the window of the parameters will be presented again and the user will be required to choose a parameter to be shown in the graph.

4. Train New Machine Learning Model-



Interactive buttons are displayed on the screen, with each button representing a distinct machine learning algorithm. Upon user selection of an algorithm, its training sequence initiates. After the training concludes, the resulting model is preserved in a PKL file within the project directory. Additionally, users have the capability to refine their training dataset by specifying a patient age range through a filter option situated beneath the algorithm selection buttons.

Show evaluation results example:

Model	Accuracy	Precision	Recall	F1-score	Filter
Decision Trees_68 to 86_model	0.36	0.39	0.36	0.37	68 to 86
Decision Trees_79 to 85_model	0.36	0.43	0.36	0.38	79 to 85
Decision Trees__model	0.38	0.43	0.38	0.39	
K-Nearest Neighbors__model	0.48	0.41	0.48	0.41	
Support Vector Machines_65.0 to 104.0_model	0.35	0.42	0.35	0.36	65.0 to 104.0

5. Generate Machine Learning Prediction-

The screenshot shows a web application window titled "Saved Models". It contains a "New Patient Data" section with three input fields: "גיל" (Age), "תאריך זמן שחרור" (Release date/time), and "משך אשפוז" (Length of stay). Below these fields is a "Save Patient" button. A message "No patient data saved" is displayed. The "Available Models" section shows a table with two columns: "Saved Model" and "Predict". The table lists two models: "decision_trees_model.pkl" and "decision_trees_68 to 102_model.pkl". Each model has a "Predict" button next to it. At the bottom, there is a "Sort by:" section with two radio buttons: "Latest Model Saved" (selected) and "Model Type".

The user required to fill the new patient's data: age, time of release from admission, and admission duration (in days).

Then, the user required to pick a model from the saved models trained.

The model will predict the class in which the patient fits best.

Prediction result for example:

The screenshot shows the prediction result for the "decision_trees_model.pkl" model. The "Predict" button is highlighted. The "Result" column displays the following information:

- Predicted class: Group 0 (1-10 days)
- Class probabilities:
 - Group 0 (1-10 days): 1.00
 - Group 1 (11-20 days): 0.00
 - Group 2 (>20 days): 0.00

As we can see, the patient in this example is most likely to be readmitted for 1-10 days.

Algorithm description

Machine Learning Algorithms

1. Decision Trees:

Algorithm: Decision trees are a supervised learning algorithm for classification and regression tasks. They recursively split the data into subsets based on the feature that best separates the data. Each split is chosen to maximize the information gain (for classification) or minimize the impurity (for regression).

Time Complexity: The time complexity of decision trees depends on the number of data points 'm', the number of features 'n', and the depth of the tree 'd'. Building a decision tree typically has a time complexity of $O(n * m * \log(m))$ or $O(n * m^2)$ for standard algorithms like CART (Classification and Regression Trees). However, decision tree algorithms can be optimized and parallelized, leading to faster training times in practice. Additionally, predicting with a decision tree typically has a time complexity of $O(\log(m))$.

2. Support Vector Machine (SVM):

Algorithm: They are used for classification and regression tasks.

In classification, SVM finds the hyperplane that best separates the classes in feature space while maximizing the margin between the classes.

In regression, SVM finds the hyperplane that best fits the data while minimizing deviations from the hyperplane.

Time Complexity: of SVM depends on the chosen kernel and the algorithm used for optimization.

For linear SVM, where no kernel trick is applied, the time complexity typically ranges from $O(n^2 * m)$ to $O(n^3 * m)$, where 'n' is the number of features and 'm' is the number of data points.

For non-linear SVM, especially with kernels like polynomial or Gaussian, the time complexity can be higher, potentially reaching $O(m^2 * n^2)$.

However, various optimization techniques and algorithms have been developed to reduce the computational cost, making SVM practical for large datasets.

3.. k-Nearest Neighbours (KNN)

Algorithm: KNN is a supervised learning algorithm for classification and regression tasks. It makes predictions based on the majority class (for classification) or the average value (for regression) of the k nearest data points in the feature space.

Time Complexity: The time complexity of the KNN algorithm can be divided into two main parts: training and prediction.

- Training: For KNN, there is no explicit training process as the algorithm stores all the training data - $O(1)$.

- Prediction: The time complexity of prediction in KNN depends on the number of data points 'm', the number of features 'n', and the value of 'k.'

For each prediction, the algorithm must compute the distances between the query point and all the training data points - $O(m * n)$, where 'm' is the number of data points and 'n' is the number of features.

However, with efficient data structures like KD-trees or Ball-trees, the prediction time complexity can be reduced to $O(\log(m) * n)$ or even $O(\log(m) * k)$ for finding the k nearest neighbors.

Data Preparation for Classification:

Our system is designed to prepare data for a three-class classification problem. The target variable, which represents the time until readmission, is categorized into three distinct groups

```
# Prepare the target variable for classification
y = y.apply(lambda w: 0 if 1 <= w <= 10 else (1 if 11 <= w <= 20 else 2))
y = y.astype('category')
```

This preprocessing step transforms the continuous time data into discrete categories for the target variable.

Additionally, the system automatically categorizes time-based features into four periods

```
# usages
@staticmethod
def categorize_time_of_day(hour):
    """Categorize the hour into numerical values representing parts of the day."""
    if pd.isnull(hour):
        return -1 # For missing values
    if 0 <= hour < 6:
        return 0 # Late Night
    elif 6 <= hour < 12:
        return 1 # Morning
    elif 12 <= hour < 18:
        return 2 # Afternoon
    elif 18 <= hour < 24:
        return 3 # Evening
    else:
        return -1 # Invalid hour
```

This categorization is applied to any input column that contains date and time information.

These preprocessing steps allow our machine learning algorithms to perform classification rather than regression.

All feature engineering and model training processes are optimized for this classification task, taking advantage of both the categorized target variable and the categorized time-based features.

System's versatility-

In accordance with the hospital's requirements, we have strived to design the system to be as versatile as possible.

If a user wishes to alter the format of one of the CSV files, they would only need to update the constant names defined at the outset.

For the system to function correctly with a new file, the following conditions must be met:

1. **Diagnosis Compatibility:** The diagnoses in the new file must correspond to those in the existing diagnoses list. Alternatively, the user can update the diagnoses in the JSON file to reflect the new terms.
2. **Date and Time Format:** The date and time in the file should adhere to the format DD/MM/YYYY HH:MM, and they should be in the same column. The regular expression for validating the date is: `r'\b\d{4}-\d{1,2}-\d{1,2}\b'`.
3. **Doctor Format:** The format for doctor entries should be "DR [NAME] M.R [NUMBER]".
4. **File Content Requirements:**
 - The new readmission file must include columns for [DOCTOR, READMISSION_DIAGNOSES].
 - The new ER file must contain columns for [ER_ARRIVAL_DATE, WALKING_ER, INTERNAL_ER, INTERNAL_ER_INF].

If the new files conform to all these constraints, the system is expected to operate correctly.

The constants in the main -

```
DIAGNOSE = "אבחנות"  
DOCTOR = "רופא מאשפז"  
SERIAL_NUMBER = "מספר סידורי מקורי"  
READMISSION_DIAGNOSES = "אבחנות בקבלה לאשפוז חוזר"  
ER_DATA = "עומס בחדר המיון"  
ER_ARRIVAL_DATE = "תאריך הגעה למיון"  
WALKING_ER = "מיון מהלכים"  
INTERNAL_ER = "מיון פנימי"  
INTERNAL_ER_INF = "רפואה דחופה זיהומים"
```

Target Variable Limitations: While our system offers flexibility in feature selection through modifiable constants, it's crucial to note that changing the target variable is not as straightforward. The current implementation is specifically designed for a three-class classification problem.

This categorization is hardcoded in the data preprocessing stage.

As a result, while users can change which column is used as the target by modifying the relevant constant, they cannot easily alter the categorization scheme or switch to a different type of prediction task (such as regression) without significant code modifications.

Any changes to the nature of the prediction task would require extensive adjustments to the data preprocessing, model training, and prediction components of the system.

The constants in the machine learning part-

```
self.target = 'זמן (בימים) מהשחרור הראשון ועד לאשפוז הבא'  
self.predictors = ['גיל', 'תאריך וזמן שחרור', 'משך אשפוז']
```

In case the user would like to use a different model for the prediction, the system could support this functionality if the model saved as a PKL file according to our saved model's format.

Save models-

```
def save_trained_model(self, model_key):
    if not model_key.endswith("_model"):
        model_key += "_model"
    model_filename = f"{model_key}.pkl"
    model_info = {
        'model': self.trained_model,
        'model_name': self.selected_algorithm,
        'filter': self.filter,
        'accuracies': self.training_models_page.model_results[model_key]['accuracies'],
        'precisions': self.training_models_page.model_results[model_key]['precisions'],
        'recalls': self.training_models_page.model_results[model_key]['recalls'],
        'f1_scores': self.training_models_page.model_results[model_key]['f1_scores'],
        'preprocessing_info': self.data_processing.preprocessing_info,
        'feature_order': self.data_processing.feature_order
    }
    with open(model_filename, 'wb') as file:
        pickle.dump(model_info, file)
```

as described in the image of the function, the model must contain the following parameters in order to read and use the model properly:

- Model – the trained machine learning model object.
- Model_name – the name of the selected algorithm ("Support Vector Machines", "K-Nearest Neighbors", "Decision Trees").
- Filter- the age filter, if there is no filter empty string.
- Accuracies
- Precision.
- Recalls.
- F1_score.
- Preprocessing_info - a dictionary containing information about how features were preprocessed (which features were scaled, categorized...)
- Feature_order – a list specifying the order of features used in the model, ensuring consistency when making predictions on new data.

How to run the project

To run the project, follow these steps:

1. Ensure you have an Integrated Development Environment (IDE) that supports Python. While we used PyCharm, you can choose any IDE you prefer.
2. Verify that you have an updated version of Python installed, specifically at least Python 3.9.13. You can check your Python version by opening the terminal and typing the following command:

```
python --version
```

Note: Depending on your system, you might need to use `python3 --version` if the `python` command refers to Python 2.x.

3. Check if you have all the required libraries installed. If not, you will need to install them using the following pip commands in the terminal:
 - `pip install pandas`
 - `pip install matplotlib`
 - `pip install scikit-learn`
 - `pip install numpy`
 - `pip install imbalanced-learn`
 - `pip install openpyxl`

Note: Some of the libraries listed above, such as `tkinter`, `JSON`, `os`, `re`, and `Pickle`, are part of the Python Standard Library and do not need to be installed separately.

4. Open the project in your chosen IDE and run it from the main class or script.
5. Initially, the system will prompt you to provide the readmission data file and the ER data file. **The system will not start without these two files. Once those files are loaded, the system saves the files paths.**

6. Remember to ensure that the paths to the data files are correctly specified within the project and that the files are formatted as expected by the system.

Note: the file paths for all CSV files are stored for future reference, eliminating the need for repeated selection.

Additionally, all the models (PKL files) are located in the directory of the project, ensuring they are automatically accessed by the system without the need for manual reloading.

The full instructions are on the video we filmed.

Link for the video- <https://youtu.be/8oXb8zawwwQ>

Summary and conclusions

The project aimed to develop a comprehensive system for analyzing and predicting re-hospitalization events at Meir Hospital. The focus was on creating a tool that could assist the hospital in understanding and mitigating the phenomenon of patient readmissions, which has significant implications for healthcare quality and costs.

Throughout the project, we conducted an extensive literature review to understand the factors contributing to re-hospitalization and explored how artificial intelligence (AI) could be leveraged to address this issue. We identified that readmissions are a common challenge not only in Israel but globally, with substantial financial implications for healthcare systems.

Our system was designed to be user-friendly and adaptable, allowing for easy integration and use by hospital staff without extensive technical expertise. The system's architecture was built on the Model-View-Controller (MVC) design pattern, ensuring a clear separation of concerns and facilitating future maintenance and updates.

The system's functionality includes the ability to upload patient data, generate various graphs for data visualization, and train machine learning models to predict the likelihood and timing of patient readmissions. We incorporated three machine learning algorithms—Decision Trees, Support Vector Machines (SVM), and k-Nearest Neighbours (KNN)—to provide a range of predictive capabilities.

In terms of usability, the system allows users to filter data and generate custom graphs, providing insights into specific aspects of the re-hospitalization data. Additionally, the system can train new models based on user-selected algorithms and make predictions on new patient data, offering a practical tool for decision support.

The project's main advantage lies in its customization for Meir Hospital's specific needs, offering a tailored solution that is not readily available in the market. While there are other advanced systems for data analysis and AI, our system stands out due to its focus on healthcare data and its cost-effectiveness.

In conclusion, the project successfully delivered a versatile and functional tool for re-hospitalization analysis and prediction at Meir Hospital. The system is expected to aid in identifying patterns and risk factors associated with readmissions, enabling the hospital to implement targeted interventions to improve patient outcomes and reduce costs. Future work could focus on further refining the machine learning models, expanding the system's capabilities, and conducting a comprehensive evaluation of its impact on hospital operations and patient care.

Known issues –

there are a couple of unresolved minor issues in our project:

1. The scrollbar functionality on the "Saved Models Page" isn't operating as intended. Users may find that adjusting their screen resolution is necessary for the scrollbar to function correctly.
2. During the training of the SVM (Support Vector Machine) model, the user interface may become unresponsive or appear to freeze. This is likely due to the extended duration required to train this particular model.

bibliography:

1. משרד הבריאות, "אשפוזים במחלקות הפנימיות 2005-2019", אתר משרד הבריאות, 2020. <https://www.gov.il/he/departments/publications/reports/internalmedical-departments-2005-20>
2. מערכת דוקטורס אונלי, "1 מכל 8 מאושפזים מגיע לאשפוז חוזר בבית חולים בתוך 2022, חודש", איגוד רופאי בריאות הציבור בישראל. <https://publichealth.doctorsonly.co.il/2022/11/271784/>
3. Santiago Romero-Brufau, Kirk D. Wyatt, Patricia Boyum, Mindy Mickelson, Matthew Moore, and Christy Cognetta-Rieke.
"Implementation of Artificial Intelligence-Based Clinical Decision Support to Reduce Hospital Readmissions at a Regional Hospital", National Library of Medicine, 2020.
Implementation of Artificial Intelligence-Based Clinical Decision Support to Reduce Hospital Readmissions at a Regional Hospital - PMC
4. Shuwen Wang and Xingquan Zhu, "Nationwide hospital admission data statistics and disease-specific 30-day readmission prediction", National Library Of Medicine, 2022.
<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9439279/>